Problem Reconstruction — Why a Stateless Assistant Cannot Serve

> Deliverable 1, Part 1 of the AI Engineering Handbook reconstruction track. This document states the **problem without naming the intended solution**. It establishes the bottleneck, the people affected, the constraints, and the pressure that any credible approach must absorb. Derived requirements appear separately in `first_principles.md`; the failure cases in detail in `failure_analysis.md`; the historical chain in `historical_timeline.pdf`.
>
> Claim tags: `[P:n]` paper-supported (see `sources.md`); `[A: …]` assumption to be validated by the naive baseline (Deliverable 3); `[O]` observed in practice.

---

1. Problem — what fails, for whom, under which conditions

A conversation assistant built on a large language model answers each turn by attending only to what is inside its context window at that moment. The model has no durable record of anything said before the current session, and no record that survives across sessions at all. This produces a class of failures that grows worse as the relationship with the user grows longer and more personal.

The precise failure. A user must re-state preferences, facts, and commitments every time they begin interacting again. The assistant cannot distinguish a fact stated once and meant to persist ("I only drink decaf") from a throwaway remark ("this café is nice"). When conversation history is replayed to compensate, the assistant cannot tell which past statements are still true, which were superseded, and which should never have been retained in the first place. [P:9][A: recency vs. relevance vs. importance weighting]

For whom it fails, and under which conditions:

Affected user	What fails	When it fails
End user	Repeats themselves; assistant forgets preferences, plans, constraints	Across sessions; after context-window eviction; in long sessions
Power user of assistants	Contradictory old guidance overrides current intent; commitments silently dropped	When history contains superseded or contradictory statements
Organizations deploying assistants	No auditable record of what the assistant knew or why it acted; data-handling liability	When retention, deletion, or consent must be demonstrable
Platform/tenant operator	One tenant's information can leak into another's answers if anything is shared	At every layer that stores or indexes user-derived content
System builder	Cost and latency balloon as history grows; quality degrades unpredictably	As stored content accumulates without governance

Under which conditions. The failures are *conditional*: they are masked when the assistant is used for short, self-contained, stateless queries ("what is 2+2"), and they become dominant exactly where assistants are most valuable — sustained, personal, task-oriented interaction with a returning user who expects continuity.

---

2. Importance — the consequence of leaving it unsolved

Leaving the problem unsolved has four compounding consequences:

1. **Trust collapse.** An assistant that forgets a user's explicitly stated preference appears careless. Repeated forgetting converts a convenience tool into a liability and suppresses adoption in high-stakes uses (health, finance, education, enterprise). [A: user-visible quality]
2. **Correctness failure, not just annoyance.** If the assistant acts on stale or contradictory stored information — an old dietary constraint, an old price, an outdated plan — the error is not a style issue; it is a wrong action with real consequences. The system must know *which* statement is true now, not merely retrieve *a* statement. [P:9][A: supersession handling]
3. **Unbounded operational cost.** Replaying or re-embedding an ever-growing history drives prompt-token usage, latency, and storage cost upward without bound, while adding diminishing (then negative) value as context becomes polluted. [P:5][O][A: token-budget discipline]
4. **Governing an uncontrolled accumulation.** If the assistant retains everything indiscriminately, it simultaneously stores personal data it has no business keeping and makes it retrievable on demand — creating privacy, compliance, and multi-tenant leakage exposure that cannot be managed after the fact. [A: admission + deletion + isolation]

The consequence of *not* solving it is not "an inconvenient chatbot." It is an assistant that cannot be trusted with continuity, cannot be governed, and cannot scale its cost. Every one of the four consequences will be measured explicitly in the naive baseline (Deliverable 3) and defended in the final design (Deliverable 4).

---

3. Constraints — what limits any credible approach

The problem must be solved *inside* hard constraints. Any approach that violates one of these is disqualified regardless of how clever it is.

#	Constraint	Why it binds
C1	**Token budget.** Each model call has a finite context window; content injected must fit a defined per-turn budget.	Model input is bounded [P:4]; unbounded replay is physically impossible at scale.
C2	**Latency.** p50/p95 response time must stay within interactive limits; retrieval and injection add on the critical path.	Every storage/retrieval step taxes the user-visible response.

#	Constraint	Why it binds
C3	**Cost.** Prompt tokens and storage grow with retained content; the system must not scale cost linearly with conversation length.	Business viability; per-tenant economics.
C4	**Correctness.** The *current* truth must win: superseded preferences must not override current ones; contradictions must be resolved, not replayed.	Wrong-action risk (Section 2.2).
C5	**Privacy & compliance.** PII must not be retained without basis; retention, correction, and deletion must be enforceable.	Legal and ethical duty; auditability required.
C6	**Multi-user / tenant isolation.** No user or tenant may retrieve another's content, directly or by adversarial construction.	Trust boundary; cross-tenant leakage is a non-negotiable gate in the handbook.
C7	**Reproducibility.** Behavior must be testable offline against fixed evaluation sets and seeds.	The handbook's evaluation framework requires independent verification.
C8	**Storage boundedness.** Retained content and indexes must be bounded and prunable, not monotonically growing.	Unbounded growth violates C1/C3 and governance.

These constraints interact: C4 often pulls against C1 (keeping more history improves correctness but consumes budget); C5 pulls against C3 (governance work is expensive); C6 forbids the cheapest possible implementation (a single shared store). A credible solution is one that makes these trade-offs explicit and measured, not one that ignores them.

---

4. Prior approaches — what was tried, and why each is insufficient

Five progressively richer but still insufficient approaches are analyzed in full in `failure_analysis.md`. Summarized here, each with the assumption that breaks:

#	Approach	Central assumption	Why it fails
A1	Stateless per-turn call	Each turn is self-contained; no cross-turn need	Forgets everything; continuity impossible
A2	Replay the full conversation	Whole history fits context and is always relevant	Overflows token budget; stale content pollutes; cost grows unbounded
A3	Keyword/symbolic lookup over stored notes	Literal string match captures relevance	Misses paraphrases; returns irrelevant exact matches; no ranking by importance/recency
A4	Naive dense retrieval over raw transcript	Semantic similarity of a chunk to the query is sufficient	Stores everything indiscriminately; low-value/sensitive items retrievable; no lifecycle; no supersession
A5	Prompt-level instruction ("remember this")	Instructions in the prompt create durable memory	Nothing persists; no admission judgment; no retrieval; no governance

Each approach is a *necessary step* in the history (see `historical_timeline.pdf`) but insufficient on its own: each fixes one bottleneck and exposes the next. The detailed assumption lists, failure scenarios, and violated constraints are in `failure_analysis.md`.

---

5. Failure evidence — what observation reveals each limit

Evidence is graded so the reader knows what is proven and what is a labelled assumption pending the naive baseline.

- **Statelessness is real and structural.** The context window is finite and the model has no external memory by construction; this is the founding observation of external-memory architectures. [P:3] [P:4][P:9]
- **Long-context replay degrades quality.** Injecting unbounded history dilutes attention and increases both cost and latency; retrieval-based systems exist precisely because context is a constrained resource. [P:4][P:5]
- **Retrieval quality depends on what was stored and how it was ranked.** A retrieval-only system that stores everything inherits the weaknesses of its ranking signal — single-similarity retrieval returns plausible-but-wrong content for personal, preference-laden queries. [P:5][A: single-signal ranking failures]
- **Lifecycle absence is observable.** Without consolidation, decay, or deletion, stale and contradictory statements survive indefinitely and are returned as if current. This is the specific failure Generative Agents address with reflection [P:8] and MemGPT addresses with a memory hierarchy and paging [P:9].
- **Governance failures are testable.** Without admission rules, sensitive content is stored and later retrievable; without isolation, cross-tenant leakage is possible under adversarial queries. [A: PII admission + isolation tests in the naive baseline]

Personal observation. The same failures are directly observable outside the literature, in sustained assistant use for software-engineering work [O]:

- **Loss of long-term project context.** RAG systems, agent workflows, and backend services are revisited after a day or two expecting continuity; instead, previously settled architectural decisions and trade-offs must be re-explained before work can resume. Replay only partially masks the loss — the restatement itself is wasted work.
- **Contradictory guidance.** After an architecture or technology choice is settled, a later conversation recommends something that directly conflicts with it, because the reasoning behind the earlier decision is no longer available. The assistant acts on stale context with equal confidence.
- **Repeated preference specification.** Stable preferences (coding style, documentation format, project constraints) must be restated turn after turn although nothing about them changed — friction without added value.

These repeat the pattern of §5's evidence: the failing mechanism is not single-turn quality, it is *what* persists across turns, sessions, and event horizons.

The naive baseline (Deliverable 3) will convert the `[A: …]` items into measured results on a fixed workload that includes: irrelevant and contradictory memories; changing preferences; long conversations

under a constrained budget; multiple users with similarly worded information; sensitive information that must not be retained; and cold-start cases.

---

6. Historical chain — how one bottleneck creates the next approach

The full chain is drawn in `historical_timeline.pdf`. In compressed form:

```
Implicit model context (stateless call)
   → bottleneck: no cross-turn memory
→ Full-conversation replay
   → bottleneck: unbounded cost/context; stale content pollution
→ External storage + retrieval (RAG)
   → bottleneck: stores everything; weak ranking; no lifecycle; no governance
→ Managed retention and selection
   → bottleneck: must reconcile correctness, privacy, isolation, and cost
```

Each arrow is a measured or structurally forced limit of the previous step — not a fashion change. A reader who accepts the chain must accept that the final step is *forced*, which is the entire point of the reconstruction.

---

7. Derived requirements — what any credible solution must do

The full first-principles derivation is in `first_principles.md`. The minimum capability set that emerges (each traceable to a specific failure):

1. Decide **what is worth retaining** (admission/classification).
2. Represent retained content with **type, provenance, and confidence**.
3. Store it **safely and isolated**, with retention and deletion guarantees.
4. **Retrieve and rank** by multiple justified signals (relevance, recency, importance, confidence).
5. **Construct context within a token budget** — select, order, prune.
6. **Update, consolidate, decay, or delete** over time (lifecycle).
7. **Expose decisions** through logs, traces, and evaluation outputs (observability).
8. Remain **reproducibly evaluable** offline.

Any solution — whatever its architecture — must satisfy all eight. Omitting one is a documented trade-off, not an accident.

Constraints → Minimum Capabilities

Constraints (from §3)	Required capabilities (from §7)
C1 token budget	R1 admission: decide what to retain
C2 latency	R2 representation: type · provenance · confidence
C3 cost	R3 safe isolated storage + deletion

C4 correctness	→	R4 retrieval + multi-signal ranking
C5 privacy / compliance		R5 context construction within budget
C6 tenant isolation		R6 lifecycle: update · consolidate · decay · delete
C7 reproducibility		R7 observability
C8 storage boundedness		R8 reproducible offline evaluation

Failure-derived capabilities are proven in `failure_analysis.md`; the last column of its summary table ties each approach failure to the capability that removes it.

---

8. Open questions — uncertainties that will shape the design

These are the unresolved questions the reconstruction deliberately leaves open, because answering them prematurely would assert an architecture before the problem is pinned down:

1. How is "importance" defined without supervision, and how does it trade against recency? [A: ranking calibration]
2. How is a *superseded* statement detected — by recency alone, by explicit correction, or by contradiction detection?
3. How should the token budget be allocated across stored content types (preferences vs. facts vs. episodic events)?
4. What is the smallest admission policy that prevents low-value *and* sensitive content from entering the store?
5. How do decay and deletion interact with a hard deletion guarantee (retention policy vs. graceful forgetting)?
6. How is "this memory is wrong" corrected end-to-end, including downstream behavior, without reintroducing the error? [P:8][A: error reinforcement]
7. In a multi-tenant setting, is isolation enforced at storage, at indexing, at retrieval, or all three — and what is the cost of each?
8. What evidence would *falsify* the premise that admission and ranking matter more than raw retrieval recall? (This is the naive baseline's job.)

These questions are deliberately framed as measurements to be made, not answers to be guessed. They become testable design hypotheses in Deliverable 4 and are resolved or explicitly deferred with evidence.

End of problem reconstruction. Continue to `historical_timeline.pdf`, `failure_analysis.md`, *and* `first_principles.md`.